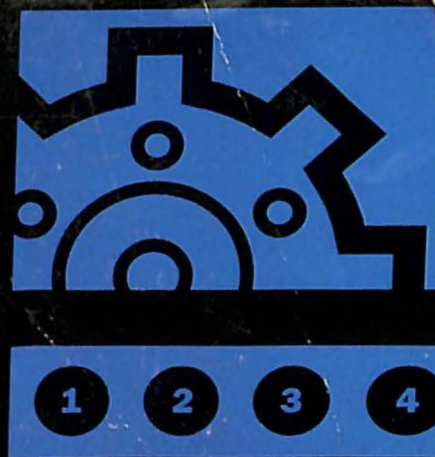
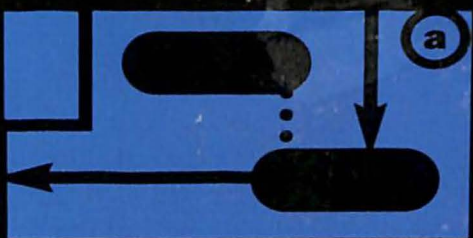
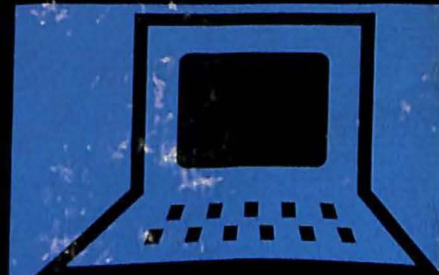
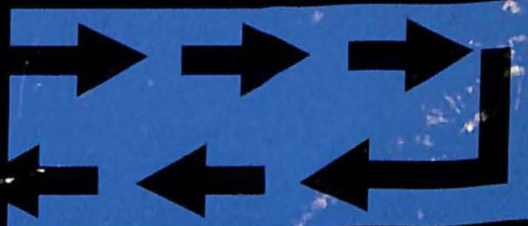


$$\Gamma_L(f) = \frac{Z_L(f)}{Z(f)}$$



Programación en Linux

C O N E J E M P L O S



```
printf("\n\n\n\n\nWhat is the  
name? "); gets(item.name);  
printf("What is the city?  
"); gets(item.city);  
getzip(item.zipcode); ans = g  
char(); if((fp=fopen(FILENAME  
"r+"))==NULL){ err_msg("R  
Read error-ensure file retu
```



Prentice
Hall

que®

Kurt Wall

KURT WALL

Programación en Linux

C O N E J E M P L O S



**BIBLIOTECA
UNIVERSIDAD de PALERMO**

Prohibida su Reproducción - Ley 11723



**Argentina • Bolivia • Brasil • Colombia • Costa Rica • Chile • Ecuador • Salvador •
España • Guatemala • Honduras • México • Nicaragua • Panamá • Paraguay • Perú •
Puerto Rico • República Dominicana • Uruguay • Venezuela**

**Amsterdam • Harlow • Londres • Menlo Park • Miami • Munich • Nueva Delhi • Nueva Jersey •
Nueva York • Ontario • París • Singapur • Sydney • Tokio • Toronto • Zurich**

datos de catalogación bibliográfica	
519.68 WAL	Wall, Kurt Programación en Linux con ejemplos - 1ª ed - Buenos Aires: Prentice Hall, 2000 568 p.; 24x19 cm Traducción de: Jorge Gorín ISBN: 987-9460-09-X I. Título - 1. Programación
UNIVERSIDAD DE PALERMO BIBLIOTECA 042613 ✓	
Fecha recepción: 08.10.03 Procedencia: COMPRA	

Editora: María Fernanda Castillo
 Armado de interior y tapa: Pérez Villamil & Asociados
 Traducción: Jorge Gorín
 Producción: Marcela Mangarelli

Traducido de:
 Linux Programming by example, by Kurt Wall, published by QUE.
 Copyright © 2000,
 All Rights Reserved.
 Published by arrangement with the original publisher,
 PRENTICE HALL, inc, A Pearson Education Company.
 ISBN: 0-7897-2215-1

Edición en Español publicada por Pearson Education, S.A.
 Copyright © 2000
 ISBN: 987-9460-09-X

Este libro no puede ser reproducido total ni parcialmente en ninguna forma, ni por ningún medio o procedimiento, sea reprográfico, fotocopia, mimeografía, informático, magnético, electroóptico, etcétera. Cualquier reproducción sin el permiso previo por escrito de la editorial viola derechos reservados, es ilegal y constituye un delito.

© 2000, PEARSON EDUCATION S.A.
 Av. Regimiento de Patricios 1959 (1266), Buenos Aires,
 República Argentina

Queda hecho el depósito que dispone la ley 11.723
 Impreso en Perú. Printed in Perú.
 Impreso en Quebecor Perú, en el mes de noviembre de 2000
 Primera edición: Diciembre de 2000

Associate Publisher
 Dean Miller

Executive Editor
 Jeff Koch

Acquisitions Editor
 Gretchen Ganser

Development Editor
 Sean Dixon

Managing Editor
 Lisa Wilson

Project Editor
 Tonya Simpsons

Copy Editor
 Kezia Endsley

Indexer
 Cheryl Landes

Proofreader
 Benjamin Berg

Technical Editor
 Cameron Laird

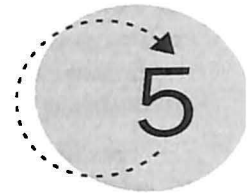
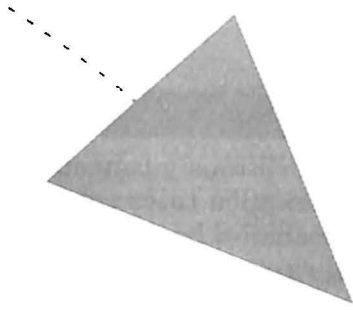
Team Coordinator
 Cindy Teeters

Interior Design
 Karen Ruggles

Cover Design
 Duane Rader

Copy Writer
 Eric Borgert

Production
 Dan Harris
 Heather Moseman



Señales

Las señales constituyen la forma más simple de comunicación entre procesos de que se dispone en Linux. Los procesos las utilizan para comunicarse entre sí, y el kernel las emplea a su vez para comunicarse con cada uno de los procesos. Las señales son también el mecanismo clave para el control de las tareas, que es la capacidad de correr aplicaciones en segundo plano o detenerlas provisoriamente. Si uno tuviese que escribir un programa de aplicación de cierta envergadura, necesitaría emplear señales.

Este capítulo cubre los siguientes temas:

- ¿Qué es una señal?
- Terminología de las señales
- Las primeras API de señales y sus problemas
- Las API de señales POSIX y Linux
- Envío y recepción de señales
- Administración de conjuntos de señales

Todos los programas de este capítulo pueden ser encontrados en el sitio Web <http://www.mcp.com/info> bajo el número de ISBN 0789722151.

Conceptos sobre señales

Cuando se habla de señales, surgen continuamente términos y conceptos diversos. Esta parte del libro define dichos términos y explica tales conceptos. A esta altura del análisis nos bastará con una comprensión básica de la terminología y de los conceptos empleados; el resto del capítulo los explicará en mayor detalle.

¿Qué es una señal?

Una señal es una pieza de software análoga a una interrupción de hardware, un suceso que tiene lugar en casi cualquier momento durante la ejecución de un proceso. Esta imprevisibilidad significa que las señales son asincrónicas. No solamente puede tener lugar una señal en cualquier momento, sino que además el proceso que recibe la señal no tiene ningún control sobre el momento en que la misma es enviada. Cada señal tiene un nombre, que comienza con SIG, como por ejemplo SIGTERM o SIGHUP. Esos nombres corresponden a constantes enteras positivas, denominadas número de señal, definidas en el encabezado del archivo de sistema `<signal.h>`.

Las señales aparecen en muchas situaciones. Una excepción de hardware, tal como una referencia ilegal a la memoria, genera una señal. También genera una señal (SIGPIPE) una excepción de software; por ejemplo, tratar de escribir a un pipe cuando el mismo no dispone de lectores (procesos que se encuentren presentes del otro lado del conducto). La función *kill* (matar) que se comenta en el capítulo 4, “Procesos”, envía una señal al proceso a ser *matado* (finalizado), lo mismo que lo hace el comando *kill*. Finalmente, las acciones generadas desde una terminal, como por ejemplo tipear Ctrl+Z en el teclado para suspender el proceso que se está ejecutando en primer plano, también generan señales.

✓ Para una exposición completa sobre cómo escribir a pipes, ver “Pipes de lectura y escritura”, página 325.

Cuando un proceso recibe una señal, puede hacer con ella una de tres cosas:

- Puede ignorar (pasar por alto) la señal.
- Puede permitir que ocurra la acción predeterminada asociada a la señal.
- Puede capturar o *interceptar* la señal, lo que hace que se ejecute una sección especial de código, denominada *handler* (manipulador) de señal. Esto se denomina precisamente manipular la señal.

Este capítulo examina cada una de estas opciones en detalle.

Terminología de las señales

Se genera una señal determinada para un proceso cada vez que ocurre algún suceso que dé lugar a esa señal, como por ejemplo una excepción de hardware. A la inversa, se dice que una señal es entregada cuando el proceso al cual ésta ha sido enviada toma acción sobre la misma. Durante el intervalo que transcurre entre la generación de una señal y su posterior entrega, se la considera pendiente. La entrega de una señal se puede *bloquear*, o sea demorar. La señal estará demorada hasta que sea desbloqueada o hasta que el proceso receptor

decida ignorarla. La manera en que un proceso responde a una señal se denomina *disposición* de la señal. Un proceso puede alternativamente pasar por alto una señal, permitir que tenga lugar su acción predeterminada, o manipularla, donde esto último significa ejecutar, en respuesta a la misma, un trozo de código personalizado. Una señal bloqueada también es considerada pendiente.

Un *conjunto de señales* es un tipo de datos de C, `sigset_t`, definido en `<signal.h>`, que es capaz de representar múltiples señales. Finalmente, la *máscara de señales* de un proceso es el conjunto de señales cuya entrega el mismo tiene bloqueado en un momento dado.

Una y otra vez se leerá en este libro que Linux, aunque haya sido diseñado siguiendo las especificaciones POSIX, ha incorporado liberalmente características de las dos ramas principales de sus antecesores, los UNIX de AT&T y BSD. Esta herencia se hace evidente en las señales. Lo que ocurre es que tanto en el UNIX de AT&T como en el de BSD se ha adoptado también una API de señal compatible con POSIX, de modo que las diferencias entre ambas API no resultan demasiado significativas (siempre y cuando todo el mundo escriba su código para la API compatible con POSIX, por supuesto). A continuación mostramos, luego de un rápido repaso del desarrollo de la API de señal, cómo enviar, interceptar y manipular señales.

Historia de las señales

Las señales han sido parte de UNIX casi desde el principio, pero a los creadores del UNIX les requirió no obstante un par de intentos lograr hacerlas funcionar correctamente. Sin entrar a considerar de lleno todos los detalles intrincados de su desarrollo, con la aparición de las primeras implementaciones de señales surgieron tres problemas principales. El primer problema fue que los manipuladores de señal tenían que volver a ser instalados cada vez que se los utilizaba, lo que daba lugar a una posible (o probable) condición de competencia. Si se entregaba una segunda señal mientras la primera estaba siendo manipulada, y antes de que se pudiese volver a instalar el manipulador de señales, o bien esta segunda señal se perdía o, alternativamente, el proceso permitía que tuviera lugar la acción original de la señal, sin ningún procesamiento.

El segundo problema ocurrido con las implementaciones iniciales de las señales fue que éstas no proveían una manera sencilla de suspender temporalmente un proceso hasta que arribase una señal. Como resultado, podía darse el caso de que se entregase una señal a un proceso y éste no cayese en cuenta de su arribo. Finalmente, las llamadas al sistema no se reiniciaban automáticamente cuando eran interrumpidas por una señal. En consecuencia, se imponía una enorme carga sobre los programadores. Después de cada llamada al sistema, éstos tenían que verificar la variable `errno` y reemitir dicha llamada si el valor obtenido para `errno` fuera `EINTR`. Las implementaciones de señales que adolecen de estos defectos se denominan señales no confiables.

La API de señales POSIX que cubre este capítulo es considerada confiable porque ha subsanado estas limitaciones. Bajo POSIX, los manipuladores de señal permanecen instalados, evitándose así la condición de competencia y sus consecuencias posteriores. Ciertas llamadas al sistema se reinician automáticamente, aliviando así la tarea del programador, y POSIX también pro-

vee una manera confiable de detener un proceso hasta que se le entregue una señal, eliminando así el problema de las señales perdidas.

Señales disponibles

La tabla 5.1 enumera todas las señales que admite Linux.

Tabla 5.1. Señales de Linux

Señal	Descripción	Acción predeterminada
SIGABRT	Generada por la función <code>abort</code> del sistema (POSIX)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)
SIGALRM	Una señal despertadora (<i>timer signal</i>) generada por la función <code>alarm</code> del sistema (POSIX)	Proceso termina
SIGBUS	El proceso trató de utilizar memoria mal alineada o sin alinear (4.2 BSD)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)
SIGCHLD	Un proceso hijo se ha detenido	Ignorarla o terminado (POSIX)
SIGCONT	El proceso debe continuar si está detenido (POSIX)	Continuar (ignorar si el proceso no está detenido)
SIGEMT	Error de bus (<i>hardware</i>)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)
SIGFPE	Excepción de punto flotante (POSIX)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)
SIGHUP	Proceso recibió un corte de línea en su terminal de control o su proceso de control murió	Proceso termina
SIGILL	Comando ilegal (POSIX)	Proceso termina y vuelca (graba) una imagen de la memoria (<i>core dump</i>)
SIGINFO	Lo mismo que SIGPWR	Ignorarla
SIGINT	Usuario generó una interrupción de teclado (POSIX)	Proceso termina
SIGIO	Se recibió E/S asincrónica (4.2 BSD)	Ignorarla
SIGIOT	Error de E/S. Igual que SIGABRT (4.2 BSD)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)
SIGKILL	Terminar proceso (POSIX)	Proceso termina, no se puede interceptar o ignorar
SIGLOST	Proceso perdió la traba de un archivo	Proceso termina
SIGPIPE	Proceso trató de escribir a un conducto sin lectores (POSIX)	Proceso termina
SIGPOLL	Ocurrió suceso encuestable (Sistema V)	Proceso termina
SIGPROF	Alarma de contorno asignada sobre un segmento de código expiró (4.2 BSD)	Proceso termina

continúa

Tabla 5.1. Señales de Linux (continuación)

Señal	Descripción	Acción predeterminada
SIGPWR	Sistema detectó un problema de suministro eléctrico (Sistema V)	Ignorarla
SIGQUIT	Usuario generó una salida desde el teclado (POSIX)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)
SIGSEGV	Proceso trató de referenciar memoria	Proceso termina y graba una imagen de la memoria (<i>core dump</i>) inaccesible (POSIX)
SIGSTKFLT	Proceso generó una falla en la pila ("derrumbó la pila")	Proceso termina
SIGSTOP	Señal de detención (POSIX)	Detención, no se puede interceptar o ignorar
SIGSYS	Argumento incorrecto pasado a rutina	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)
SIGTERM	Proceso recibió una	Proceso termina señal de terminación (POSIX)
SIGTRAP	Proceso llegó a un punto de detención (generalmente durante una depuración de código) (POSIX)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)
SIGTSTP	Usuario generó una detención desde el teclado (POSIX)	Detención
SIGTTIN	Proceso trató de leer desde <code>stdin</code> mientras corría en segundo plano (POSIX)	Detención
SIGTTOU	Proceso intentó escribir a <code>stdout</code> mientras corría en segundo plano (POSIX)	Detención
SIGUNUSED	Señal no utilizada	Proceso termina
SIGURG	Condición de urgencia en un socket (4.2 BSD)	Ignorarla
SIGUSR1	Señal 1 definida por el usuario (POSIX)	Proceso termina
SIGUSR2	Señal 2 definida por el usuario (POSIX)	Proceso termina
SIGVTALRM	Alarma de intervalos expiró (4.2 BSD)	Proceso termina
SIGWINCH	El tamaño de una ventana terminal cambió (4.3 BSD, Sun)	Ignorarla
SIGXCPU	Proceso excedió la cantidad de tiempo de CPU que podía utilizar (4.2 BSD)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)

continúa

Tabla 5.1. Señales de Linux (continuación)

Señal	Descripción	Acción predeterminada
SIGXFSZ	Proceso trató de acceder o manipular un archivo mayor que el límite de tamaño de archivo del sistema (4.2 BSD)	Proceso termina y graba una imagen de la memoria (<i>core dump</i>)

Como se puede ver a partir de la tabla 5.1, las señales que Linux reconoce son una mezcla de señales derivadas de BSD, de Sistema V o de AT&T y, por supuesto, de POSIX. Varias de las señales listadas, sin embargo, incluyendo SIGEMT, SIGCLD, SIGINFO y SIGLOST, no están implementadas (¡por lo que el hecho de que no obstante se encuentren documentadas en la página 7 (señales) del manual constituye un misterio!). Los siguientes párrafos tratan la mayoría de estas señales más detalladamente.

SIGABRT es generada por la función `abort` que se comenta en el capítulo 4. SIGALRM y SIGVTALRM se generan cuando los temporizadores (*timers*) configurados utilizando las llamadas `alarm` y `setitimer`, respectivamente, las disparan. La llamada `alarm` se comenta más adelante en este mismo capítulo en la parte “Establecimiento de una alarma.” SIGBUS de hecho no tiene lugar en los sistemas Linux; cuando un proceso trata de utilizar una memoria desalineada, el kernel, en lugar de generar SIGBUS, repara él mismo el problema de alineación y luego continúa. SIGCHLD se envía a un proceso padre cuando uno de sus procesos hijos termina o se detiene, lo que permite al padre llamar a una de las funciones `wait` para obtener el estado de salida.

✓ Las funciones `wait` se explican detalladamente en “Esperas en procesos”, página 84.

SIGHUP tiene lugar cuando termina un líder de sesión o cuando se cierra una terminal de control. SIGFPE se envía cuando ocurre cualquier excepción aritmética, tal como *overflows* (desbordes), *underflows* (desbordes por la derecha) o divisiones por cero. SIGILL es generada por otra excepción de hardware, la ejecución de un comando ilegal. SIGINT hace que todos los procesos presentes en el grupo de procesos que está ejecutándose en primer plano terminen porque el usuario generó una interrupción desde el teclado, generalmente tipeando Ctrl+C.

SIGQUIT, de manera similar, se genera cuando se tipea desde el teclado el carácter de salida, generalmente Ctrl+\. Otra señal generada desde el teclado, Ctrl+Z, genera SIGTSTP. Las funciones SIGKILL y SIGTERM son generadas por la función `kill`. Obsérvese que SIGKILL no puede ser interceptada ni ignorada; esta configuración permite disponer al superusuario de un método no ambiguo para matar un proceso que no se esté comportando adecuadamente.

SIGSTOP detiene cualquier proceso y, lo mismo que SIGKILL, no puede ser interceptada ni ignorada. Sin embargo, a diferencia de SIGKILL, SIGSTOP meramente detiene un proceso que se esté ejecutando. La ejecución del mismo puede continuarse enviándole SIGCONT. Lamentablemente, no existe algo así como SIGRESURRECT para los procesos eliminados. SIGTTIN y SIGTTOU tienen lugar cuando un proceso que se está ejecutando en segundo plano intenta obtener entrada desde su terminal de control o escribir a la misma. Cuando una ventana de terminal cambia de tamaño, todos los proce-

sos que se están ejecutando en primer plano reciben la señal SIGWINCH. Las señales SIGUSR1 y SIGUSR2 están reservadas para uso particular de cada proceso. SIGXCPU y SIGXFS, finalmente, se generan cuando un proceso se excede de su tiempo asignado de CPU o de su máximo tamaño permitido de archivos, respectivamente.

Envío de señales

Desde un programa, existen dos maneras de enviar una señal a un proceso en ejecución: utilizando el comando `kill` (`kill(1)`) o empleando la función `kill(2)`. El comando `kill` es en realidad una interfaz de usuario hacia la función `kill`.

✓ Para obtener una descripción detallada de la función `kill`, ver “Eliminación (*killing*) de procesos”, pág. 86.

Empleo del comando `kill`

Para poder utilizar el comando `kill` en un programa, se debe llamar a una de estas tres funciones: `system`, `fork` o `exec`. Como se ha visto en el capítulo anterior, las llamadas a `system` o a `fork` generan un nuevo proceso que tendrá a su cargo la ejecución de `kill`, mientras que `exec` reemplaza al proceso que efectuó la llamada antes de ejecutar `kill`. El resultado, sin embargo, es el mismo: el proceso correspondiente queda terminado.



EJEMPLO

Ejemplo

Este ejemplo primero genera un nuevo proceso que no hace otra cosa que permanecer inactivo. Luego una llamada a `exec` elimina el proceso hijo antes de que el programa finalice.

```
/* Nombre del programa en Internet: pkill.c */
/*
 * eliminar_proceso.c - Enviar una señal por medio de kill(1)
 */
#include <sys/types.h>
#include <wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    pid_t hijo;
    char cadena[6]; /* Cinco dígitos de un PID mas un dígito para el cero de
terminacion */

    if((hijo = fork()) < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if(hijo == 0) { /* El programa se encuentra en el proceso hijo */
        sleep(30);
    } else {
```

```

/* El programa se encuentra en el proceso padre, */
/* de modo que matara al proceso hijo */
sprintf(cadena, "%d", hijo);
printf("Eliminando %s\n", cadena);
if((execl("/bin/kill", "/bin/kill", cadena, NULL)) < 0) {
    /* El exec fracaso, de modo que esperar */
    /* y obtener la condicion de salida */
    perror("execl");
    waitpid(hijo, NULL, 0);
    exit(EXIT_FAILURE);
}
}
exit(EXIT_FAILURE); /* El programa no deberia llegar hasta aqui */
}

```

Después de verificar que el fork tuvo efectivamente lugar, el proceso hijo queda inactivo (“duerme”) durante 30 segundos, tiempo más que suficiente para que el padre lo elimine. Si, por alguna razón, la llamada a `exec` fracasara, resulta importante aguardar hasta que el proceso hijo finalice, de modo de poder obtener su condición de salida. Recuérdese del capítulo anterior que las funciones de `exec` nunca regresan un código de salida al programa que las llamó, a menos que tenga lugar un error. El programa también utiliza la función `sprintf` para convertir el PID numérico del hijo en una cadena alfanumérica terminada con un cero que pueda ser pasada sin problemas a la función `execl`. Cuando se corre el programa, la salida simplemente indica que el padre está eliminando a su hijo:



SALIDA

```
$ ./ eliminar_proceso
```

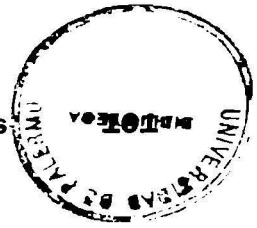
```
Eliminando 759
```

Por supuesto, el PID indicado sea probablemente distinto para el sistema con que cuente el lector.

Empleo de la función `kill`

El empleo de la función `kill` es más sencillo que la ejecución del comando `kill`, porque no se deben realizar los pasos adicionales para preparar una cadena para `exec`. Todo lo que se requiere es el PID y la señal que se desea utilizar. La razón para utilizar `execl` en el ejemplo anterior fue simplemente poder ilustrar el uso de `/bin/kill` en un programa en ejecución.

Se habrá notado que ninguna señal tiene el valor 0. La única que tiene este valor es la *señal nula*, que tiene un propósito especial. Si uno le transfiere a `kill` la señal nula, `kill` no envía ninguna señal, sino que se remite a realizar su verificación rutinaria de errores. Esto último puede ser útil si se desea comprobar si un proceso determinado se encuentra todavía ejecutándose, empleando su PID para buscarlo. Téngase en cuenta, sin embargo, que los PIDs se reutilizan periódicamente, de modo que, en una máquina atareada, éste no constituye un método confiable de comprobar la existencia de un determinado proceso.

**EJEMPLO****Ejemplo**

El siguiente ejemplo utiliza la función `kill` para enviar dos señales a un proceso hijo inactivo, una de las cuales será ignorada y la otra eliminará dicho proceso.

```
/* Nombre del programa en Internet: fkill.c */
/*
 * matahijo.c - Enviar una señal por medio de kill(2)
 */
#include <sys/types.h>
#include <wait.h>
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    pid_t hijo;
    int codigo_de_retorno;
    if((hijo = fork()) < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if(hijo == 0) {          /* El programa se encuentra en el proceso hijo */
        sleep(30);
    } else { /* El programa se encuentra en el proceso padre*/
        /* Enviar una señal que sea ignorada */
        printf("Enviando SIGCHLD a %d\n", hijo);
        codigo_de_retorno = kill(hijo, SIGCHLD);
        if(codigo_de_retorno < 0)
            perror("kill:SIGCHLD");
        else
            printf("El proceso %d todavia existe\n", hijo);
        /* Ahora hacer terminar al proceso hijo */
        printf("Eliminando %d\n", hijo);
        if((kill(hijo, SIGTERM)) < 0)
            perror("kill:SIGTERM");
        /* esperar para obtener la condicion de salida */
        waitpid(hijo, NULL, 0 );
    }
}
```

```

    }
    exit(EXIT_SUCCESS);
}

```

Esta es la salida que se obtiene de un par de ejecuciones de este programa:



SALIDA

```

$ ./matahijo
Enviando SIGCHLD a 871
El proceso 871 todavía existe
Eliminando 871
$ ./matahijo
Enviando SIGCHLD a 879
El proceso 879 todavía existe
Eliminando 879

```

La primera cosa que se debe advertir es que, aunque parezca extraño, `kill` (en inglés, matar) puede ser utilizada para enviar señales diferentes a las que son necesarias para matar un proceso (`SIGKILL`, `SIGTERM`, `SIGQUIT`). Segundo, el proceso hijo ignoró, de hecho, a `SIGCHLD`. Finalmente, dado que mi sistema se encontraba relativamente inactivo, pude utilizar la señal nula para confirmar que el proceso al que se le envió la señal todavía existía. Verificado esto último, `SIGTERM` terminó este proceso hijo.

La llamada a `waitpid`, insisto, constituye una medida de seguridad para el caso de que la función `kill` no tenga éxito. Como se observó en el capítulo anterior, no existe manera de saber por anticipado si el proceso hijo terminará antes o después del proceso padre. Si el que terminase primero fuera el proceso padre, el proceso hijo se convertiría en un huérfano adoptado por `init`, e `init` obtendría la condición de salida del mismo. Si fuera el proceso hijo el que terminase primero, sin embargo, debe ser el proceso padre quien obtenga su condición de salida para evitar que aquel se convierta en un proceso zombie que ocupe innecesariamente un lugar en la tabla de procesos del kernel.

Intercepción de señales

El aspecto engorroso del envío de señales lo constituye la intercepción y la manipulación de las mismas. Cada proceso puede decidir cómo responder a todas las señales excepto `SIGSTOP` y `SIGKILL`, las cuales, como se vio en la tabla 5.1, no pueden ser ni interceptadas ni ignoradas. La manera más simple de interceptar señales no consiste de hecho en interceptarlas sino en simplemente esperar a que sean enviadas. La función `alarm` pone en funcionamiento un temporizador (*timer*) que envía la señal `SIGALRM` cuando el intervalo de tiempo a que fue ajustado este último expira. La función `pause` se comporta de manera similar, excepto porque lo que hace es suspender la ejecución de un proceso hasta que el mismo reciba alguna señal.

Programación de una alarma

La función alarma, cuyo prototipo se encuentra en `<unistd.h>`, activa un temporizador en el proceso que efectúa la llamada. Cuando el tiempo asignado al temporizador expira, se envía `SIGALRM` al proceso que efectuó la llamada y, a menos que este último intercepte la señal, la acción predeterminada para `SIGALRM` consiste en dar fin a dicho proceso.

El prototipo para `alarm` es:

```
unsigned int alarm(unsigned int seconds);
```

`seconds` es el número de segundos de reloj después de los cuales el temporizador expira. El valor que retorna esta función es 0 si no se ha programado ninguna otra alarma, o el número de segundos remanente en una alarma programada previamente, si la hubiera. Un proceso puede tener sólo una alarma. Estableciendo el número de segundos en 0 se cancela cualquier alarma previamente programada.



EJEMPLO

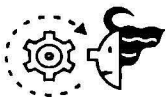
Ejemplos

1. Este programa establece una alarma de cinco segundos, después de lo cual termina.

```
/* Nombre del programa en Internet: mysleep.c */
/*
 * alarma.c - Implementacion sencilla de sleep(1)
 */
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    /* Ajustar la alarma */
    if((alarm(5)) > 0)
        puts("Ya hay una alarma programada");
    /* Dejar inactivo el proceso el tiempo suficiente */
    /* como para que la alarma expire */
    sleep(30);
    /* El programa no deberia llegar aqui */
    puts("&Como fue que el programa llego aqui?");
    exit(EXIT_FAILURE);
}
```

La salida de este programa es la siguiente:



SALIDA

```
$ ./alarma
Alarm clock
```

Este programa establece una alarma de cinco segundos, observa si ya se había establecido alguna otra alarma, y luego queda inactivo durante 30 segundos para proveer a la alarma de una razonable cantidad de tiempo para expirar. En sistemas atareados puede transcurrir un tiempo adicional entre la generación de `SIGALRM` y su llegada (aunque no 25 segundos, por supuesto). Cuando arriba la señal `SIGALRM`, alarma termina. El mensaje "Alarm clock" (reloj despertador) lo generó el kernel, no el programa alarma. Por esa razón el mismo aparece en inglés.



EJEMPLO

2. Este ejemplo establece dos alarmas, llamando a `alarm` una tercera vez para cancelar la segunda alarma.

```

/* Nombre del programa en Internet: 2alarm.c */
/*
 * dos_alarmas.c - Cancelacion de alarmas
 */
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    long int codigo_de_retorno;

    /* Establecer la alarma */
    if((alarm(15)) > 0)
        puts("Ya se encuentra establecida una alarma");
    /* Establecer una nueva alarma */
    sleep(3);
    codigo_de_retorno = alarm(5);
    if(codigo_de_retorno > 0)
        printf("Restan %ld segundos en la primera alarma\n", codigo_de_retorno);
    /* Cancelar la segunda alarma */
    sleep(2);
    printf("Restan %d segundos en la segunda alarma\n", alarm(0));

    exit(EXIT_FAILURE);
}

```

La salida de este programa es la siguiente:



SALIDA

```

$ ./2alarm
Restan 12 segundos en la primera alarma
Restan 3 segundos en la segunda alarma

```

Como el programa canceló la segunda alarma antes de que ésta tuviera la oportunidad de expirar, al transferirle 0 como argumento correspondiente a segundos, el programa no exhibió el mensaje "Alarm clock". Reitero, en un sistema muy cargado, las alarmas pueden arribar más tarde de lo previsto. Una razón por la cual uno podría llegar a utilizar alarmas es para establecer un límite de tiempo para los programas potencialmente largos, tales como los que deben clasificar un archivo muy largo o esperar que un usuario

responda a un mensaje. La mayoría de los programas que establecen alarmas también las interceptan, en lugar de permitirles terminar el proceso.

Utilización de la función pause

La función `pause` suspende el proceso que la llama hasta que arribe alguna señal. El proceso que efectúa la llamada debe de estar en condiciones de manipular la señal que recibe, o en caso contrario tendrá lugar la acción predefinida de la misma. `pause` se encuentra prototipada en `<unistd.h>`:

```
int pause(void);
```

`pause` retorna al proceso que la llamó sólo si el proceso intercepta una señal. Si la señal que se recibe invoca un handler, éste se ejecutará antes de que `pause` retorne. `pause` siempre retorna `-1` y asigna a `errno` el valor `EINTR`.



EJEMPLO

Ejemplo

Este sencillo programa sólo espera el arribo de una señal y luego finaliza.

```
/* Nombre del programa en Internet: pause.c */
/*
 * pausa.c - Efectua una pausa y luego termina
 */
#include <unistd.h>
#include <stdlib.h>

int main(void)
{
    pause();
    exit(EXIT_SUCCESS);
}
```

La salida de este programa es la siguiente:



SALIDA

```
$ ./pause
[Ctrl+z]

[1]+  Stopped                  ./pause

fg
./pause
[Ctrl+\]
Quit (core dumped)

$ ./pause
User defined signal 1
```

En la primera corrida, `pause` sólo produce una suspensión indefinida. Primero, `Ctrl+Z` (ingresada desde el teclado), suspende el programa; el comando `fg` lo vuelve a traer al primer plano, para que entonces `Ctrl+\` (`SIGQUIT`)

lo elimine y ocasione un vuelco de memoria. Si no se lo hubiese eliminado, hubiera continuado en pausa hasta que recibiera otra señal o hasta que el sistema se reiniciara.

Durante la segunda corrida, el programa corrió en una ventana. Su PID se obtuvo en una segunda ventana por medio de `ps`, y luego se emitió `kill -USR1 <PID>`. Como no se estableció un handler de señal para la señal `SIGUSR1`, ésta llevó a cabo su acción predeterminada de terminación.

Definición de un handler de señales

En algunos casos, el comportamiento deseado de la señal es justamente su acción predeterminada. En otras, probablemente la mayoría de las veces, se desea modificar ese comportamiento, o llevar a cabo tareas adicionales. En estos casos, se debe definir e instalar un handler personalizado de señal que modifique el comportamiento predeterminado.

Consideremos el caso de un proceso padre que engendra varios procesos hijos. Cuando los procesos hijos terminan, el proceso padre recibe la señal `SIGCHLD`. Para efectuar un seguimiento adecuado de sus procesos hijos y supervisar su condición de salida, el proceso padre puede ya sea llamar a `wait`, inmediatamente después de engendrar cada proceso hijo o, de manera más eficiente, establecer un handler de señal que llame a `wait` (o a `waitpid`) cada vez que al mismo le llegue `SIGCHLD`.

POSIX define un conjunto de funciones para crear y manipular señales. El procedimiento general consiste en crear un conjunto de señales, establecer las señales que se desea interceptar, registrar un handler de señales en el kernel y aguardar a interceptar la señal.

LA API DE ADMINISTRACIÓN DE SEÑALES

Para crear, establecer e interrogar a un conjunto de señales, utilice las cinco funciones siguientes, todas ellas definidas en `<signal.h>`:

- `int sigemptyset(sigset_t *set);`
- `int sigfillset(sigset_t *set);`
- `int sigaddset(sigset_t *set, int signum);`
- `int sigdelset(sigset_t *set, int signum);`
- `int sigismember(const sigset_t *set, int signum);`

`set` es un conjunto de señales de tipo `sigset_t`, como se explicó al comienzo del capítulo. `sigemptyset` inicializa el conjunto de señales `set` de modo que queden excluidas del mismo todas las señales. `sigfillset`, a la inversa, inicializa `set` de manera tal que estén incluidas en él todas las señales. `sigaddset` añade a `set` la señal `signum`. `sigdelset` elimina `signum` de `set`.

Estas cuatro funciones retornan `0` cuando tienen éxito o `-1` si ocurre un error. Finalmente, `sigismember` comprueba si `signum` se encuentra en `set`, retornando `1` (verdadero) si la misma se encuentra presente o `0` (falso) si no lo está.

CREACIÓN DE UN CONJUNTO DE SEÑALES

Para crear un conjunto de señales, pues, utilice `sigemptyset` o `sigfillset` para inicializar un conjunto de señales. Si crea un conjunto de señales vacío, necesitará emplear `sigaddset` para agregarle las señales en las que se encuentre interesado. Si por el contrario crea un conjunto que comprenda todas las señales, utilice `sigdelset` para eliminar del mismo las señales que no desee.

**EJEMPLO****Ejemplos**

1. Este programa añade una señal a un conjunto de señales vacío, y luego utiliza `sigismember` para confirmar que la señal se encuentra presente en el conjunto.

```
/* Nombre del programa en Internet: mkset.c */
/*
 * crear_conjunto_señales.c - Bloquea un conjunto de señales
 */
#include <signal.h>
#include <stdlib.h>
#include <stdio.h>

void salir_si_error(char *);

int main(void)
{
    sigset_t nuevo_conjunto;

    /* Crear el conjunto */
    if((sigemptyset(&nuevo_conjunto)) < 0)
        salir_si_error("sigemptyset");
    /* Añadir SIGCHLD al conjunto */
    if((sigaddset(&nuevo_conjunto, SIGCHLD)) < 0)
        salir_si_error("sigaddset");

    /* Verificar el nuevo conjunto de señales resultante */
    if(sigismember(&nuevo_conjunto, SIGCHLD))
        puts("SIGCHLD esta presente en el conjunto de señales");
    else
        puts("SIGCHLD no esta presente en el conjunto de señales");
    /* SIGTERM no deberia ser parte de dicho conjunto */
    if(sigismember(&nuevo_conjunto, SIGTERM))
        puts("SIGTERM esta presente en el conjunto de señales");
    else
        puts("SIGTERM no esta presente en el conjunto de señales");
}
```



```

        exit(EXIT_SUCCESS);
    }

    void salir_si_error(char *nombre_señal)
    {

        perror(nombre_señal);
        exit(EXIT_FAILURE);

    }

```

**SALIDA**

```

$ ./crear_conjunto_señales
SIGCHLD esta presente en el conjunto de señales
SIGTERM no esta presente en el conjunto de señales

```

crear_conjunto_señales primero crea un conjunto de señales vacío, transfiriéndole a sigemptyset la dirección de una estructura sigset_t. Luego añade SIGCHLD dicho conjunto de señales. Finalmente utilizando sigismember, confirma que SIGCHLD es efectivamente parte del conjunto de señales y que SIGTERM no es parte del mismo.

**EJEMPLO**

2. Este ejemplo ilustra el proceso de remoción de una señal desde un conjunto de señales.

```

/* Nombre del programa en Internet: rmset.c */
/*
 * eliminar_señales.c - Elimina señales de un conjunto de señales
 */
#include <signal.h>
#include <stdlib.h>
#include <stdio.h>

void salir_si_error(char *);

int main(void)
{
    sigset_t nuevo_conjunto;

    /* Crear el conjunto */
    if((sigfillset(&nuevo_conjunto)) < 0)
        salir_si_error("sigfillset");
    /* Eliminar SIGALRM del conjunto */
    if((sigdelset(&nuevo_conjunto, SIGALRM)) < 0)
        salir_si_error("sigaddset");

```

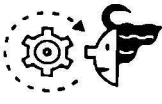
```

/* SIGALRM debería haber sido eliminada */
if(sigismember(&nuevo_conjunto, SIGALRM))
    puts("SIGALRM esta presente en el conjunto de señales");
else
    puts("SIGALRM no esta presente en el conjunto de señales");
/* SIGTERM debería seguir estando presente */
if(sigismember(&nuevo_conjunto, SIGTERM))
    puts("SIGTERM esta presente en el conjunto de señales");
else
    puts("SIGTERM no esta presente en el conjunto de señales");

exit(EXIT_SUCCESS);
}

void salir_si_error(char *nombre_señal)
{
    perror(nombre_señal);
    exit(EXIT_FAILURE);
}

```

**SALIDA**

```

$ ./eliminar_señales
SIGALRM no esta presente en el conjunto de señales
SIGTERM esta presente en el conjunto de señales

```

`eliminar_señales` se comporta a la inversa del programa del ejemplo anterior, creando un conjunto que incluye la totalidad de las señales. Luego elimina SIGALRM. De nuevo, la llamada a `sigismember` confirma que la señal eliminada ha sido efectivamente suprimida y que las demás señales siguen estando presentes.

REGISTRO DEL HANDLER

El mero hecho de crear un conjunto de señales y luego agregar o eliminar señales del mismo, sin embargo, no representa crear un handler de señales ni permite interceptar o bloquear señales. Hay otros pasos adicionales que se deben adoptar. Primero, se debe utilizar `sigprocmask` para establecer o modificar (o ambas cosas a la vez) la respectiva máscara de señales corriente; si no se hubiese aún establecido una máscara de señales, todas las señales darán lugar a una acción predeterminada. Una vez que se establezca una máscara de señal, se deberá registrar un handler para la señal o señales que se desea interceptar, por medio de `sigaction`.

Como sería de esperar a esta altura del texto, `sigaction` y `sigprocmask` se encuentran prototipadas en `<signal.h>`. Sus respectivos prototipos son los siguientes:

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

`sigprocmask` establece o examina la máscara de señales en vigencia, y lo hace según el valor de `how`, el cual puede ser uno de los tres siguientes:

- `SIG_BLOCK` - `set` contiene señales adicionales a ser bloqueadas.
- `SIG_UNBLOCK` - `set` contiene señales que deberán ser desbloqueadas.
- `SIG_SETMASK` - `set` contiene la nueva máscara de señal.

Si `how` equivale a `NULL`, será ignorado. Si `set` vale `NULL`, la máscara corriente se almacena en `oldset`; si `oldset` equivale a `NULL`, será ignorada. `sigprocmask` retorna 0 si tiene éxito y -1 si encuentra algún error.

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

`sigaction` activa el handler de señal para la señal especificada en `signum`. La estructura `struct sigaction` describe la manera de disponer de la señal. Su definición completa, presente en (tal cual) `<signal.h>`, es:

```
struct sigaction {
    void (*sa_handler)(int);
    sigset_t sa_mask;
    int sa_flags;
    void (*sa_restorer)(void);
};
```

`sa_handler` es un puntero a una función que especifica el handler, o función, a ser invocados cuando sea generada la señal especificada en `signum`. La función debe de estar definida con un valor de retorno de tipo `void` (es decir, no retornará ningún valor) y debe de aceptar un argumento de tipo `int`. Alternativamente, el argumento de `sa_handler` puede ser también `SIG_DFL`, lo que hace que tenga lugar la acción predeterminada de `signum`, o `SIG_IGN`, lo que ocasionará que esta señal sea ignorada.

Mientras un handler de señal se encuentra en ejecución, la señal que lo activó queda bloqueada. `sa_mask` define la máscara de un conjunto de señales adicionales que deberían quedar bloqueadas durante la ejecución del handler. `sa_flags` es una máscara que modifica el comportamiento de `sa_handler`'s. Puede ser una o más de las siguientes alternativas:

- `SA_NOCLDSTOP`. El proceso ignorará cualquier señal `SIGSTOP`, `SIGTSTP`, `SIGTTIN` y `SIGTTOU` generada por el proceso hijo.
- `SA_ONESHOT` o `SA_RESETHAND`. El handler de señales personalizado se ejecutará sólo una vez. Luego de que se ejecute, se restaurará la acción predeterminada de la señal.
- `SA_RESTART`. Habilita la posibilidad de efectuar llamadas a sistema reiniciables.
- `SA_NOMASK` o `SA_NODEFER`. No se opone a que la señal sea recibida en el interior de su propio handler.

No le preste atención al elemento `sa_restorer`; el mismo está obsoleto y no debe ser empleado. Tenga en cuenta también que la expresión `sigaction` corresponde tanto al nombre de una función como al del patrón de una estructura asociada a la misma.

En resumen, `sigprocmask` manipula el conjunto de señales que se desea bloquear e interroga a la máscara de señales en vigencia. La función `sigaction` registra en el kernel un handler de una o más señales y configura el comportamiento exacto del mismo.

Ejemplos

1. El primer ejemplo sencillamente bloquea `SIGALRM` y `SIGTERM` sin proceder a instalar un handler que adopte alguna acción especial.

```
/* Nombre del programa en Internet: block.c. En ese listado es incorrecto */
/* el nombre asignado al programa y la descripcion de su accion */
/*
 * bloquear_señales.c - Bloquea un conjunto de señales
 */
#include <signal.h>
#include <stdlib.h>
#include <stdio.h>

void salir_si_error(char *);

int main(void)
{
    sigset_t nuevo_conjunto;

    /* Crear el conjunto */
    if((sigemptyset(&nuevo_conjunto)) < 0)
        salir_si_error("sigemptyset");
    /* Agregar SIGTERM y SIGALRM */
    if((sigaddset(&nuevo_conjunto, SIGTERM)) < 0)
        salir_si_error("sigaddset:SIGTERM");
    if((sigaddset(&nuevo_conjunto, SIGALRM)) < 0)
        salir_si_error("sigaddset:SIGALRM");

    /* Bloquear las señales sin manipularlas */
    if((sigprocmask(SIG_BLOCK, &nuevo_conjunto, NULL)) < 0)
        salir_si_error("sigprocmask");

    /* Aguardar el arribo de una señal */
    pause();

    exit(EXIT_SUCCESS);
}
```

```

    }
    void salir_si_error(char *mensaje)
    {
        perror(mensaje);
        exit(EXIT_FAILURE);
    }

```

**SALIDA**

Una corrida de prueba de este programa produjo la siguiente salida. El programa se ejecutó en una ventana y se le enviaron señales por medio del comando kill desde otra ventana (los comandos enviados desde la segunda ventana se muestran entre corchetes).

```

$ ./bloquear_señales
[$ kill -TERM $(pidof ./bloquear_señales)]
[$ kill -ALRM $(pidof ./bloquear_señales)]
[$ kill -QUIT $(pidof ./bloquear_señales)]
Quit (core dumped)

```

Como se puede observar a partir de dicha salida, el envío de los procesos SIGTERM y SIGALRM no produjo ningún efecto, aunque la acción predeterminada de ambas señales es terminar el proceso que las recibe. El comando pidof ./bloquear_señales retorna el PID asociado con ./bloquear_señales, mientras que los resultados del comando son sustituidos por la construcción \$(...) y transferidos al comando kill que, por lo tanto, no produce efecto. En cambio cuando el proceso recibe SIGQUIT termina, como lo muestra la salida. Nótese que dado que el programa bloquea a SIGTERM y SIGALRM, pause nunca retorna porque el proceso no recibe la señal.

**EJEMPLO**

2. Este programa establece un handler para SIGUSR1. La acción predeterminada de SIGUSR1 es terminar el proceso. El handler personalizado sólo indica que ha sido llamado.

```

/* Nombre del programa en Internet: blkusr.c */
/*
 * bloquear_usuario.c - Handler personalizado para SIGUSR1
 */
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
void salir_si_error(char *); /* funcion de error */
void handler_usuario1(int); /* handler de señales */
int main(void)
{

```

```

struct sigaction accion;

/* Configurar el handler */
accion.sa_handler = handler_usuario1;
sigemptyset(&accion.sa_mask);
accion.sa_flags = SA_NOCLDSTOP;

/* Registrar el handler */
if((sigaction(SIGUSR1, &accion, NULL)) < 0)
    salir_si_error("sigaction");
/* Permitir el suficiente tiempo como para enviar una señal */
sleep(60);

exit(EXIT_SUCCESS);
}

void salir_si_error(char *mensaje)
{
    perror(mensaje);
    exit(EXIT_FAILURE);
}

void handler_usuario1(int signum)
{
    if(signum == SIGUSR1)
        puts("Interceptada USR1");
    else
        printf("Interceptada %d\n", signum);
}

```

handler_usuario1 es un sencillo handler de señal, que no hace más que informar el hecho de que interceptó a SIGUSR1. Como el miembro sa_mask de accion, la estructura de patrón sigaction, debería contener señales adicionales a las cuales bloquear mientras se ejecuta el handler de señal, uno simplemente lo inicializa con un conjunto vacío de señales transfiriéndolo directamente a sigemptyset. Una vez que el handler se encuentra configurado, sigaction lo registra para SIGUSR1. Como en este caso no interesa la respuesta anterior del handler, se asigna NULL como valor de oset. Finalmente, el programa queda inactivo durante 60 segundos como para disponer del suficiente tiempo como para enviarle una señal. Corra el programa en una ventana, y envíele SIGUSR1 desde otra, como se ilustra:



SALIDA

```
$ ./bloquear_usuario
[kill -USR1 $(pidof ./ bloquear_usuario)]
Interceptada SIGUSR1
```

La salida muestra que el handler fue registrado exitosamente. Luego de ejecutarse el handler, el programa termina.

CONSEJO

SIGUSR1 y SIGUSR2 han sido especialmente reservadas como señales definidas por el programador. Se las debe emplear cuando se desea habilitar algún tipo de comportamiento especial, tal como volver a leer un archivo de configuración, en lugar de utilizar un handler de señal a fin de redefinir una de las señales estándar, cuya semántica ya se encuentra bien definida.

Detección de señales pendientes

`sigpending` hace posible que un proceso detecte *señales pendientes* (señales que fueron generadas mientras las mismas se encontraban bloqueadas) y luego decidir si ignorarlas o permitir que lleven a cabo su acción. ¿Para qué se debe verificar qué señales están pendientes? Supongamos que el lector desea escribir a un archivo, y dicha operación no debe ser interrumpida a fin de preservar la integridad del archivo. Durante el proceso de escritura al disco, pues, uno deseará mantener bloqueadas SIGTERM y SIGQUIT, pero fuera de esta circunstancia las desea administrar o permitirles que lleven a cabo su acción predeterminada. De modo que, antes de dar comienzo a la operación de escritura, uno bloqueará a SIGTERM y SIGQUIT. Una vez que la escritura a archivo se haya completado satisfactoriamente, se debería verificar la existencia de señales pendientes y, si SIGTERM o SIGQUIT se encontrasen pendientes, se necesitaría desbloquearlas. O, sencillamente se las podrá desbloquear sin molestarse en verificar si se encuentran pendientes o no. Verificar la presencia de señales pendientes o no es opcional. Si se desea ejecutar un bloque especial de código si se encuentra presente cierta señal, verifíquela. En otro caso, simplemente desbloquéelas.

Igual que las demás funciones de señales, `sigpending` se encuentra prototipada en `<signal.h>`. El prototipo de `sigpending` 's es:

```
int sigpending(sigset_t *set);
```

El conjunto de las señales pendientes se retorna en `set`. La llamada en sí retorna 0 si resultó exitosa o -1 si se registró algún error. Utilice `sigismember` a fin de determinar si las señales en las que se encuentra interesado están pendientes, es decir, si se encuentran presentes en `set`.



EJEMPLO

Ejemplo

Para propósitos de demostración, este programa bloquea a SIGTERM, luego determina si se encuentra pendiente y finalmente la ignora y termina normalmente.

```
/* Nombre del programa en Internet: pending.c */
/*
 * verificar_pendientes.c - Divertimento con sigpending
 */
#include <sys/types.h>
```

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    sigset_t conjunto_señales, conjunto_pendientes;
    struct sigaction accion;

    sigemptyset(&conjunto_señales);

    /* Añadir la señal de interes */
    sigaddset(&conjunto_señales, SIGTERM);

    /* Bloquear la señal */
    sigprocmask(SIG_BLOCK, &conjunto_señales, NULL);

    /* Enviar SIGTERM hacia el propio proceso */
    kill(getpid(), SIGTERM);

    /* Obtener señales pendientes */
    sigpending(&conjunto_pendientes);

    /* Si SIGTERM se encuentra pendiente, ignorarla */
    if(sigismember(&conjunto_pendientes, SIGTERM)) {
        sigemptyset(&accion.sa_mask);
        accion.sa_handler = SIG_IGN; /* Ignorar SIGTERM */
        sigaction(SIGTERM, &accion, NULL);
    }

    /* Desbloquear SIGTERM */
    sigprocmask(SIG_UNBLOCK, &conjunto_señales, NULL);

    exit(EXIT_SUCCESS);
}
```

En beneficio de la brevedad se ha omitido la verificación de errores. El código resulta sencillo de interpretar. El mismo crea una máscara que bloquea a SIGTERM y luego utiliza kill para enviar SIGTERM hacia sí mismo. Como está bloqueada, la señal no llega a destino. Se utiliza sigpending y sigis-

`member` para determinar si `SIGTERM` se encuentra pendiente y, si fuese así, para configurar su acción a `SIG_IGN`. Cuando las señales se desbloquean, `SIGTERM` se ignora y el programa termina normalmente.

Lo que viene

Habiendo adquirido un conocimiento básico de la administración de procesos y el procesamiento de señales, el lector se encuentra listo para desplazarse hacia temas más avanzados, tales como E/S sencilla de archivos y a administración más sofisticada de archivos, tal como *multiplexing* y bloqueo de archivos. *El multiplexing de E/S*, que es el proceso de leer y escribir simultáneamente desde y hacia múltiples fuentes de entrada y salida, requiere tener un buen conocimiento sobre señales. Los *daemons*, procesos que corren en segundo plano sin intervención del usuario, dependen tanto de la capacidad de realizar *forks* (generar procesos descendientes) y de verificar múltiples procesos de entrada y salida como de la interceptación de señales que sean de interés y que provengan de los procesos hijos, y de actuar en consecuencia. Primero, sin embargo, el próximo capítulo explica las *llamadas a sistema*, la interfaz programática entre los programas de aplicación y los servicios provistos por el kernel.